

My Project

1

Generated by Doxygen 1.9.7

1 Namespace Index	1
1.1 Namespace List	1
2 Class Index	3
2.1 Class List	3
3 File Index	5
3.1 File List	5
4 Namespace Documentation	7
4.1 MLWENO Namespace Reference	7
4.1.1 Detailed Description	7
4.2 tensorProductPoly Namespace Reference	7
5 Class Documentation	9
5.1 tensorProductPoly::basePolynomial Class Reference	9
5.1.1 Constructor & Destructor Documentation	9
5.1.1.1 basePolynomial() [1/3]	9
5.1.1.2 basePolynomial() [2/3]	10
5.1.1.3 basePolynomial() [3/3]	10
5.1.1.4 ~basePolynomial()	10
5.1.2 Member Function Documentation	10
5.1.2.1 der() [1/2]	10
5.1.2.2 der() [2/2]	10
5.1.2.3 eval() [1/2]	10
5.1.2.4 eval() [2/2]	11
5.1.2.5 getCoef() [1/2]	11
5.1.2.6 getCoef() [2/2]	11
5.1.2.7 getI()	11
5.1.2.8 getJ()	11
5.1.2.9 getSize()	11
5.1.2.10 operator>() [1/2]	11
5.1.2.11 operator>() [2/2]	12
5.1.2.12 printCoef()	12
5.1.2.13 setCoef()	12
5.1.2.14 setMaxDegree()	12
5.2 MLWENO::MLWENOPrepate Class Reference	12
5.2.1 Detailed Description	13
5.2.2 Constructor & Destructor Documentation	13
5.2.2.1 MLWENOPrepate()	13
5.2.2.2 ~MLWENOPrepate()	13
5.2.3 Member Function Documentation	13
5.2.3.1 AddLevel()	13
5.2.3.2 PrintInfo()	13

5.2.3.3 UpdateDerivSmoothnessIndic()	14
5.2.3.4 UpdateSmoothnessIndic()	14
5.2.4 Friends And Related Symbol Documentation	14
5.2.4.1 multiLevelReconstruction	14
5.3 MLWENO::multiLevelReconstruction Class Reference	14
5.3.1 Constructor & Destructor Documentation	15
5.3.1.1 multiLevelReconstruction() [1/4]	15
5.3.1.2 multiLevelReconstruction() [2/4]	16
5.3.1.3 multiLevelReconstruction() [3/4]	16
5.3.1.4 multiLevelReconstruction() [4/4]	16
5.3.1.5 ~multiLevelReconstruction()	16
5.3.2 Member Function Documentation	16
5.3.2.1 AddLevel() [1/3]	16
5.3.2.2 AddLevel() [2/3]	17
5.3.2.3 AddLevel() [3/3]	17
5.3.2.4 AssignboundaryCells()	17
5.3.2.5 AssignboundaryLevels()	17
5.3.2.6 AssigninteriorCells()	18
5.3.2.7 AssigninteriorLevels()	18
5.3.2.8 Clear()	18
5.3.2.9 EvaluateDerivMLWENO()	18
5.3.2.10 EvaluateDerivMLWENOAdd()	18
5.3.2.11 EvaluateMLWENO()	19
5.3.2.12 GetboundaryCells()	19
5.3.2.13 GetboundaryLevels()	19
5.3.2.14 GetInfo()	19
5.3.2.15 GetinteriorCells()	19
5.3.2.16 GetinteriorLevels()	20
5.3.2.17 GetScale() [1/2]	20
5.3.2.18 GetScale() [2/2]	20
5.3.2.19 ModifyReconstMethod() [1/2]	20
5.3.2.20 ModifyReconstMethod() [2/2]	20
5.3.2.21 PrintBoundaryLayer()	20
5.3.2.22 PrintNonLinearWgts()	21
5.3.2.23 PrintSmoothnessIndicator()	21
5.3.2.24 SelectWenoReconstLevel() [1/2]	21
5.3.2.25 SelectWenoReconstLevel() [2/2]	21
5.3.2.26 SeparateBoundaryLayer() [1/3]	21
5.3.2.27 SeparateBoundaryLayer() [2/3]	22
5.3.2.28 SeparateBoundaryLayer() [3/3]	22
5.3.2.29 UpdateNonLinearWgts() [1/2]	22
5.3.2.30 UpdateNonLinearWgts() [2/2]	22

5.3.2.31 UpdateNonLinearWgtsAndDerivs()	22
5.4 MLWENO::singleLevelReconstruction Class Reference	23
5.4.1 Constructor & Destructor Documentation	23
5.4.1.1 singleLevelReconstruction() [1/2]	23
5.4.1.2 singleLevelReconstruction() [2/2]	24
5.4.1.3 ~singleLevelReconstruction()	24
5.4.2 Member Function Documentation	24
5.4.2.1 CalculateSmoothnessIndic()	24
5.4.2.2 CheckExist()	24
5.4.2.3 CheckStencilPolynomials()	24
5.4.2.4 CheckStencils()	25
5.4.2.5 CreateStencilPolynomials()	25
5.4.2.6 Evaluate() [1/2]	25
5.4.2.7 Evaluate() [2/2]	25
5.4.2.8 GetScale() [1/2]	25
5.4.2.9 GetScale() [2/2]	25
5.4.2.10 GetSizeX()	26
5.4.2.11 GetSizeY()	26
5.4.2.12 GetSmoothnessIndic()	26
5.4.2.13 GetSmoothnessIndicDeriv()	26
5.4.2.14 PrintSmoothnessIndicator()	26
5.4.2.15 UpdateDerivSmoothnessIndic()	26
5.4.2.16 UpdateSmoothnessIndic()	27
5.5 stencil< T > Class Template Reference	27
5.5.1 Constructor & Destructor Documentation	27
5.5.1.1 stencil() [1/4]	28
5.5.1.2 stencil() [2/4]	28
5.5.1.3 stencil() [3/4]	28
5.5.1.4 stencil() [4/4]	28
5.5.1.5 ~stencil()	28
5.5.2 Member Function Documentation	28
5.5.2.1 clearPtr()	28
5.5.2.2 CreateStencil()	29
5.5.2.3 getI()	29
5.5.2.4 getJ()	29
5.5.2.5 getK()	29
5.5.2.6 getSize()	29
5.5.2.7 GetStencil()	29
5.5.2.8 operator()() [1/6]	29
5.5.2.9 operator()() [2/6]	30
5.5.2.10 operator()() [3/6]	30
5.5.2.11 operator()() [4/6]	30

5.5.2.12 operator>() [5/6]	30
5.5.2.13 operator>() [6/6]	30
5.5.2.14 setI()	31
5.5.2.15 setJ()	31
5.5.2.16 setK()	31
5.5.2.17 SetStencil() [1/3]	31
5.5.2.18 SetStencil() [2/3]	31
5.5.2.19 SetStencil() [3/3]	31
5.6 tensorProductPoly::stencilPolynomial Class Reference	32
5.6.1 Constructor & Destructor Documentation	32
5.6.1.1 stencilPolynomial() [1/3]	32
5.6.1.2 stencilPolynomial() [2/3]	32
5.6.1.3 stencilPolynomial() [3/3]	33
5.6.1.4 ~stencilPolynomial()	33
5.6.2 Member Function Documentation	33
5.6.2.1 eval() [1/4]	33
5.6.2.2 eval() [2/4]	33
5.6.2.3 eval() [3/4]	33
5.6.2.4 eval() [4/4]	33
5.6.2.5 GetDerivSmoothIndic()	34
5.6.2.6 GetOrderX()	34
5.6.2.7 GetOrderY()	34
5.6.2.8 GetScale()	34
5.6.2.9 GetSmoothIndic()	34
5.6.2.10 operator>() [1/4]	34
5.6.2.11 operator>() [2/4]	34
5.6.2.12 operator>() [3/4]	35
5.6.2.13 operator>() [4/4]	35
5.6.2.14 printCoef() [1/2]	35
5.6.2.15 printCoef() [2/2]	35
5.6.2.16 SetCollapsePolyn()	35
5.6.2.17 SetStencilPolynomials()	35
5.6.2.18 SetUpScale() [1/2]	36
5.6.2.19 SetUpScale() [2/2]	36
6 File Documentation	37
6.1 /home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp File Reference	37
6.2 /home/renpo/Research/MantleSolver/src/mlweno/polynomial.h File Reference	37
6.3 polynomial.h	38
6.4 /home/renpo/Research/MantleSolver/src/mlweno/reconstruction.cpp File Reference	39
6.5 /home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h File Reference	39
6.6 reconstruction.h	40

6.7 /home/renpo/Research/MantleSolver/src/mlweno/stencil.h File Reference	43
6.7.1 Enumeration Type Documentation	43
6.7.1.1 stencilType	43
6.8 stencil.h	43
Index	45

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

MLWENO	
Weno reconstruction with a given stencil level	7
tensorProductPoly	7

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

tensorProductPoly::basePolynomial	9
MLWENO::MLWENOPrepare	12
MLWENO::multiLevelReconstruction	14
MLWENO::singleLevelReconstruction	23
stencil< T >	27
tensorProductPoly::stencilPolynomial	32

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

/home/renpo/Research/MantleSolver/src/mlweno/ polynomial.cpp	37
/home/renpo/Research/MantleSolver/src/mlweno/ polynomial.h	37
/home/renpo/Research/MantleSolver/src/mlweno/ reconstruction.cpp	39
/home/renpo/Research/MantleSolver/src/mlweno/ reconstruction.h	39
/home/renpo/Research/MantleSolver/src/mlweno/ stencil.h	43

Chapter 4

Namespace Documentation

4.1 MLWENO Namespace Reference

Weno reconstruction with a given stencil level.

Classes

- class [MLWENOPrepare](#)
- class [multiLevelReconstruction](#)
- class [singleLevelReconstruction](#)

4.1.1 Detailed Description

Weno reconstruction with a given stencil level.

4.2 tensorProductPoly Namespace Reference

Classes

- class [basePolynomial](#)
- class [stencilPolynomial](#)

Chapter 5

Class Documentation

5.1 tensorProductPoly::basePolynomial Class Reference

```
#include <polynomial.h>
```

Public Member Functions

- [basePolynomial](#) ()
- [basePolynomial](#) (const int maxDegree[2])
- [basePolynomial](#) (const int maxDegree[2], double *coef)
- [~basePolynomial](#) ()
- void [setMaxDegree](#) (const int maxDegree[2])
- void [setCoef](#) (double *coef)
- double * [getCoef](#) () const
- int [getSize](#) () const
- int [getI](#) () const
- int [getJ](#) () const
- double [eval](#) (double x, double y) const
- double [eval](#) (vertex P) const
- double [operator\(\)](#) (double x, double y) const
- double [operator\(\)](#) (vertex P) const
- double [der](#) (int derX, int derY, double x, double y) const
- double [der](#) (int derX, int derY, vertex P) const
- void [printCoef](#) () const
- double [getCoef](#) (int i) const

5.1.1 Constructor & Destructor Documentation

5.1.1.1 basePolynomial() [1/3]

```
tensorProductPoly::basePolynomial::basePolynomial ( ) [inline]
```

5.1.1.2 basePolynomial() [2/3]

```
basePolynomial::basePolynomial (
    const int maxDegree[2] )
```

5.1.1.3 basePolynomial() [3/3]

```
basePolynomial::basePolynomial (
    const int maxDegree[2],
    double * coef )
```

5.1.1.4 ~basePolynomial()

```
basePolynomial::~~basePolynomial ( )
```

5.1.2 Member Function Documentation**5.1.2.1 der() [1/2]**

```
double tensorProductPoly::basePolynomial::der (
    int derX,
    int derY,
    double x,
    double y ) const
```

5.1.2.2 der() [2/2]

```
double tensorProductPoly::basePolynomial::der (
    int derX,
    int derY,
    vertex P ) const [inline]
```

5.1.2.3 eval() [1/2]

```
double basePolynomial::eval (
    double x,
    double y ) const
```

5.1.2.4 eval() [2/2]

```
double tensorProductPoly::basePolynomial::eval (
    vertex P ) const [inline]
```

5.1.2.5 getCoef() [1/2]

```
double * basePolynomial::getCoef ( ) const
```

5.1.2.6 getCoef() [2/2]

```
double basePolynomial::getCoef (
    int i ) const
```

5.1.2.7 getI()

```
int tensorProductPoly::basePolynomial::getI ( ) const [inline]
```

5.1.2.8 getJ()

```
int tensorProductPoly::basePolynomial::getJ ( ) const [inline]
```

5.1.2.9 getSize()

```
int tensorProductPoly::basePolynomial::getSize ( ) const [inline]
```

5.1.2.10 operator>() [1/2]

```
double tensorProductPoly::basePolynomial::operator() (
    double x,
    double y ) const [inline]
```

5.1.2.11 operator() [2/2]

```
double tensorProductPoly::basePolynomial::operator() (
    vertex P ) const [inline]
```

5.1.2.12 printCoef()

```
void basePolynomial::printCoef ( ) const
```

5.1.2.13 setCoef()

```
void basePolynomial::setCoef (
    double * coef )
```

5.1.2.14 setMaxDegree()

```
void basePolynomial::setMaxDegree (
    const int maxDegree[2] )
```

The documentation for this class was generated from the following files:

- /home/renpo/Research/MantleSolver/src/mlweno/[polynomial.h](#)
- /home/renpo/Research/MantleSolver/src/mlweno/[polynomial.cpp](#)

5.2 MLWENO::MLWENOPrepare Class Reference

```
#include <reconstruction.h>
```

Public Member Functions

- [MLWENOPrepare](#) ()
- [~MLWENOPrepare](#) ()
- void [AddLevel](#) (const MeshInfo &mi, const int &stencilSizeX, const int &stencilSizeY)
- void [UpdateSmoothnessIndic](#) (const MeshInfo &mi)
- void [UpdateDerivSmoothnessIndic](#) (const MeshInfo &mi)
- void [PrintInfo](#) ()

Friends

- class [multiLevelReconstruction](#)

5.2.1 Detailed Description

A class preparing for all possible multi-level weno reconstrucion using different single-level reconstruction. This class holding all single level reconstructions. Let [MLWENO](#) "steal" single level reconstructions from it. Multi-level reconstruction class is declared as a friend of this class.

5.2.2 Constructor & Destructor Documentation

5.2.2.1 MLWENOPrepare()

```
MLWENO::MLWENOPrepare::MLWENOPrepare ( ) [inline]
```

5.2.2.2 ~MLWENOPrepare()

```
MLWENOPrepare::~~MLWENOPrepare ( )
```

5.2.3 Member Function Documentation

5.2.3.1 AddLevel()

```
void MLWENOPrepare::AddLevel (
    const MeshInfo & mi,
    const int & stencilSizeX,
    const int & stencilSizeY )
```

Add single levels to the private allLevels_ member. Reconstruction method not required. Reconstruction method will be defined later in [multiLevelReconstruction](#). Create single level key

Check if the new level has never been defined before

Initialize single level reconstruction class pointer.

Create map from single level key to created single level reconstrucion

5.2.3.2 PrintInfo()

```
void MLWENOPrepare::PrintInfo ( )
```

Print information of all levels created. Print added levels and reconstruction methods

5.2.3.3 UpdateDerivSmoothnessIndic()

```
void MLWENOPrepare::UpdateDerivSmoothnessIndic (
    const MeshInfo & mi )
```

Update derivatives of smoothness indicator for all single level stencil polynomials. Should be called before using fully implicit method.

5.2.3.4 UpdateSmoothnessIndic()

```
void MLWENOPrepare::UpdateSmoothnessIndic (
    const MeshInfo & mi )
```

Update smoothness indicator for all single level stencil polynomials. Should be called everytime non linear weights are being calculated.

5.2.4 Friends And Related Symbol Documentation

5.2.4.1 multiLevelReconstruction

```
friend class multiLevelReconstruction [friend]
```

The documentation for this class was generated from the following files:

- [/home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h](#)
- [/home/renpo/Research/MantleSolver/src/mlweno/reconstruction.cpp](#)

5.3 MLWENO::multiLevelReconstruction Class Reference

```
#include <reconstruction.h>
```

Public Member Functions

- [multiLevelReconstruction](#) ()
A constructor.
- [multiLevelReconstruction](#) (const MeshInfo &mi, int stencilSizeX, int stencilSizeY, vector< indice > brm)
- [multiLevelReconstruction](#) (const MeshInfo &mi, int *stencilSize, vector< indice > brm)
- [multiLevelReconstruction](#) (const MeshInfo &mi, vector< int * > stencilSizes, vector< vector< indice > > brms)
- [~multiLevelReconstruction](#) ()
A destructor.
- void [AddLevel](#) (const MeshInfo &mi, int stencilSizeX, int stencilSizeY, vector< indice > brm)
Add a single weno reconstruction level to the computation.
- void [AddLevel](#) (const MeshInfo &mi, int *stencilSize, vector< indice > brm)
- void [AddLevel](#) (const MeshInfo &mi, vector< int * > stencilSizes, vector< vector< indice > > brms)

- void [SeparateBoundaryLayer](#) (const MeshInfo &mi, const int &layerSize)
Specify boundary layers (cells near boundary that need additional reconstruction level then interior cells)
- void [SeparateBoundaryLayer](#) (const MeshInfo &mi)
- void [SeparateBoundaryLayer](#) (const MeshInfo &mi, const int &layerSize, const unordered_set< std::string > &additionalLevels)
- const unordered_set< int > & [GetboundaryCells](#) () const
Get a copy of calculated boundary and interior cells.
- const unordered_set< int > & [GetinteriorCells](#) () const
- void [AssignboundaryCells](#) (const unordered_set< int > &boundaryCells)
Assigning calculated boundary and interior cells.
- void [AssigninteriorCells](#) (const unordered_set< int > &interiorCells)
- const unordered_set< std::string > & [GetboundaryLevels](#) () const
Get a copy of calculated boundary and interior levels.
- const unordered_set< std::string > & [GetinteriorLevels](#) () const
- void [AssignboundaryLevels](#) (const unordered_set< std::string > &boundaryLevels)
Assigning calculated boundary and interior levels.
- void [AssigninteriorLevels](#) (const unordered_set< std::string > &interiorLevels)
- void [UpdateNonLinearWgts](#) (const MeshInfo &mi, const int stage)
- void [UpdateNonLinearWgts](#) (const MeshInfo &mi)
- void [UpdateNonLinearWgtsAndDerivs](#) (const MeshInfo &mi)
- void [ModifyReconstMethod](#) (std::string key, vector< indice > newReconstMethod)
- void [ModifyReconstMethod](#) (int stencilSizeX, int stencilSizeY, vector< indice > newReconstMethod)
- void [SelectWenoReconstLevel](#) (const unordered_set< std::string > &keys)
- void [SelectWenoReconstLevel](#) (const unordered_set< std::string > &keys, const [MLWENOPrepare](#) &mlpPtr)
- double [EvaluateMLWENO](#) (const MeshInfo &mi, vertex point, indice cell) const
- unordered_map< int, double > [EvaluateDerivMLWENO](#) (const MeshInfo &mi, const vertex &point, const indice &global) const
- unordered_map< int, double > [EvaluateDerivMLWENOAdd](#) (const MeshInfo &mi, const vertex &point, const indice &global) const
- const double [GetScale](#) (std::string level, int s)
- const double [GetScale](#) (std::string level, const MeshInfo &mi, const indice &owner)
- void [GetInfo](#) ()
Routines used to check results and verification.
- void [PrintBoundaryLayer](#) (const MeshInfo &mi)
- void [PrintSmoothnessIndicator](#) (const MeshInfo &mi)
- void [PrintNonLinearWgts](#) (const MeshInfo &mi)
- void [Clear](#) ()
Clear created multi level reconstruction.

5.3.1 Constructor & Destructor Documentation

5.3.1.1 multiLevelReconstruction() [1/4]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction ( ) [inline]
```

A constructor.

! Construt multi-level weno reconstruction by specifying each single level

5.3.1.2 multiLevelReconstruction() [2/4]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    int stencilSizeX,
    int stencilSizeY,
    vector< indice > brm ) [inline]
```

Be careful with the parametrized constructor function. It is better to "steal" from [MLWENOPrepare](#), instead of calculate itself.

5.3.1.3 multiLevelReconstruction() [3/4]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    int * stencilSize,
    vector< indice > brm ) [inline]
```

5.3.1.4 multiLevelReconstruction() [4/4]

```
MLWENO::multiLevelReconstruction::multiLevelReconstruction (
    const MeshInfo & mi,
    vector< int * > stencilSizes,
    vector< vector< indice > > brms ) [inline]
```

5.3.1.5 ~multiLevelReconstruction()

```
MLWENO::multiLevelReconstruction::~~multiLevelReconstruction ( ) [inline]
```

A destructor.

Construct multi-level weno reconstruction by specifying each single level

5.3.2 Member Function Documentation

5.3.2.1 AddLevel() [1/3]

```
void MLWENO::multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    int * stencilSize,
    vector< indice > brm ) [inline]
```


5.3.2.2 AddLevel() [2/3]

```
void multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    int stencilSizeX,
    int stencilSizeY,
    vector< indice > brm )
```

Add a single weno reconstruction level to the computation.

Add a single weno reconstruction level to the weno reconstruction. Create a reference key name for specific level. Create map from the reference key to the single level reconstruction. Create single level key

Check if the new level has never been defined before

Insert new key into allLevels;

Initialize single level reconstruction class pointer.

Create map from single level key to created single level reconstruction

Create map from single level key to reconstruction method (how to find stencils)

Update lowest and highest order of reconstruction level

Accumulate total levels

5.3.2.3 AddLevel() [3/3]

```
void MLWENO::multiLevelReconstruction::AddLevel (
    const MeshInfo & mi,
    vector< int * > stencilSizes,
    vector< vector< indice > > brms ) [inline]
```

5.3.2.4 AssignboundaryCells()

```
void MLWENO::multiLevelReconstruction::AssignboundaryCells (
    const unordered_set< int > & boundaryCells ) [inline]
```

Assing calculated boundary and interior cells.

5.3.2.5 AssignboundaryLevels()

```
void MLWENO::multiLevelReconstruction::AssignboundaryLevels (
    const unordered_set< std::string > & boundaryLevels ) [inline]
```

Assing calculated boundary and interior levels.

5.3.2.6 AssignInteriorCells()

```
void MLWENO::multiLevelReconstruction::AssignInteriorCells (
    const unordered_set< int > & interiorCells ) [inline]
```

5.3.2.7 AssignInteriorLevels()

```
void MLWENO::multiLevelReconstruction::AssignInteriorLevels (
    const unordered_set< std::string > & interiorLevels ) [inline]
```

5.3.2.8 Clear()

```
void multiLevelReconstruction::Clear ( )
```

Clear created multi level reconstruction.

Clear everything. Delete all pointers defined inside the map. Can be called directly. May crash if destructor is called after [Clear\(\)](#) is called.

5.3.2.9 EvaluateDerivMLWENO()

```
unordered_map< int, double > multiLevelReconstruction::EvaluateDerivMLWENO (
    const MeshInfo & mi,
    const vertex & point,
    const indice & global ) const
```

Derivative of the reconstruction of value with respect to the given point with multi level weno method.

Derivative of the reconstruction of value with respect to the given point with multi level weno method. Start with pseudo derivative where non linear weights are not differentiated. Extract nonlinear weights linked to the target cell.

Evaluate in the multi level weno fashion.

5.3.2.10 EvaluateDerivMLWENOAdd()

```
unordered_map< int, double > multiLevelReconstruction::EvaluateDerivMLWENOAdd (
    const MeshInfo & mi,
    const vertex & point,
    const indice & global ) const
```

Differentiate non linear weight when evaluate [MLWENO](#) derivative. Should be added to the original derivative in addition.

5.3.2.11 EvaluateMLWENO()

```
double multiLevelReconstruction::EvaluateMLWENO (
    const MeshInfo & mi,
    vertex point,
    indice globalCell ) const
```

Evaluation of given point with selected weno reconstruction method.

Reconstruction of a given point value with selected weno reconstruction method. Convert local cell indice to global cell indice.

Get calculated nonlinear weights linked to this cell.

Evaluate in the multi level weno fashion.

5.3.2.12 GetboundaryCells()

```
const unordered_set< int > & MLWENO::multiLevelReconstruction::GetboundaryCells ( ) const
[inline]
```

Get a copy of calculated boundary and interior cells.

5.3.2.13 GetboundaryLevels()

```
const unordered_set< std::string > & MLWENO::multiLevelReconstruction::GetboundaryLevels ( )
const [inline]
```

Get a copy of calculated boundary and interior levels.

5.3.2.14 GetInfo()

```
void multiLevelReconstruction::GetInfo ( )
```

Routines used to check results and verification.

Print added levels and reconstruction methods

5.3.2.15 GetinteriorCells()

```
const unordered_set< int > & MLWENO::multiLevelReconstruction::GetinteriorCells ( ) const
[inline]
```

5.3.2.16 GetinteriorLevels()

```
const unordered_set< std::string > & MLWENO::multiLevelReconstruction::GetinteriorLevels ( )
const [inline]
```

5.3.2.17 GetScale() [1/2]

```
const double MLWENO::multiLevelReconstruction::GetScale (
    std::string level,
    const MeshInfo & mi,
    const indice & owner ) [inline]
```

5.3.2.18 GetScale() [2/2]

```
const double MLWENO::multiLevelReconstruction::GetScale (
    std::string level,
    int s ) [inline]
```

Get scale of the selected single level reconstruction with respect to the selected cell.

5.3.2.19 ModifyReconstMethod() [1/2]

```
void MLWENO::multiLevelReconstruction::ModifyReconstMethod (
    int stencilSizeX,
    int stencilSizeY,
    vector< indice > newReconstMethod ) [inline]
```

5.3.2.20 ModifyReconstMethod() [2/2]

```
void multiLevelReconstruction::ModifyReconstMethod (
    std::string key,
    vector< indice > newReconstMethod )
```

Update reconstruction methods. Called when same level are used, but different reconstruction methods. For example, advection and diffusion reconstruction uses third order level but with different stencils. When an existing level is added, update reconstruction method with new one.

Modify reconst method with given key. Only modify existing reconst method. Please use AddLevel to add reconstruction levels.

5.3.2.21 PrintBoundaryLayer()

```
void multiLevelReconstruction::PrintBoundaryLayer (
    const MeshInfo & mi )
```

5.3.2.22 PrintNonLinearWgts()

```
void multiLevelReconstruction::PrintNonLinearWgts (
    const MeshInfo & mi )
```

5.3.2.23 PrintSmoothnessIndicator()

```
void multiLevelReconstruction::PrintSmoothnessIndicator (
    const MeshInfo & mi )
```

5.3.2.24 SelectWenoReconstLevel() [1/2]

```
void multiLevelReconstruction::SelectWenoReconstLevel (
    const unordered_set< std::string > & keys )
```

Select Weno reconstruction levels to be used in the reconstruction.

Modify wenoLevels_. Selecting predefined reconstruction method in the actual computation process. Make wenoLevels_ an empty set

Insert selected keys

5.3.2.25 SelectWenoReconstLevel() [2/2]

```
void multiLevelReconstruction::SelectWenoReconstLevel (
    const unordered_set< std::string > & keys,
    const MLWENOPrepare & mlpPtr )
```

A completely different function despite sharing the same name with the previous one. "Stealing" single level reconstruction from class [MLWENOPrepare](#).

"Stealing" prepared single level reconstruction pointer from [MLWENOPrepare](#) class.

5.3.2.26 SeparateBoundaryLayer() [1/3]

```
void multiLevelReconstruction::SeparateBoundaryLayer (
    const MeshInfo & mi )
```

5.3.2.27 SeparateBoundaryLayer() [2/3]

```
void multiLevelReconstruction::SeparateBoundaryLayer (
    const MeshInfo & mi,
    const int & layerSize )
```

Specify boundary layers (cells near boundary that need additional reconstrucion level then interior cells)

Specify boundary layers (cells near boundary that need additional reconstrucion level then interior cells) Default boundary layer size is set to be 1. Call the second function when a boundary layer size larger than 1 is needed. Create two different weno reconstruction levels for interior and boundary cells.

5.3.2.28 SeparateBoundaryLayer() [3/3]

```
void multiLevelReconstruction::SeparateBoundaryLayer (
    const MeshInfo & mi,
    const int & layerSize,
    const unordered_set< std::string > & additionalLevels )
```

A more advanced boudarylayer separation function Assign not only the (1,1) level to boundary cells. Create two different weno reconstruction levels for interior and boundary cells.

5.3.2.29 UpdateNonLinearWgts() [1/2]

```
void multiLevelReconstruction::UpdateNonLinearWgts (
    const MeshInfo & mi )
```

Update non linear weights default with two stage method Default multi level reconstruction will utilize two stage nonlinear weights

5.3.2.30 UpdateNonLinearWgts() [2/2]

```
void multiLevelReconstruction::UpdateNonLinearWgts (
    const MeshInfo & mi,
    const int stage )
```

Update non linear weights for all levels. Smoothness indicator will be updated every time when non linear weights updated.

5.3.2.31 UpdateNonLinearWgtsAndDerivs()

```
void multiLevelReconstruction::UpdateNonLinearWgtsAndDerivs (
    const MeshInfo & mi )
```

Default multi level reconstruction will utilize two stage nonlinear weights

The documentation for this class was generated from the following files:

- /home/renpo/Research/MantleSolver/src/mlweno/[reconstruction.h](#)
- /home/renpo/Research/MantleSolver/src/mlweno/[reconstruction.cpp](#)

5.4 MLWENO::singleLevelReconstruction Class Reference

```
#include <reconstruction.h>
```

Public Member Functions

- [singleLevelReconstruction](#) ()
A constructor.
- [singleLevelReconstruction](#) (int stencilSizeX, int stencilSizeY)
A constructor.
- [~singleLevelReconstruction](#) ()
A destructor.
- void [CreateStencilPolynomials](#) (const MeshInfo &mi)
- bool [CheckExist](#) (const MeshInfo &mi, indice owner) const
- const int [GetSizeX](#) () const
- const int [GetSizeY](#) () const
- const double [GetScale](#) (int s)
- const double [GetScale](#) (const MeshInfo &mi, const indice &owner)
- double [CalculateSmoothnessIndic](#) (const MeshInfo &mi, indice owner)
Calculate smoothness indicator for the single stencil.
- double [GetSmoothnessIndic](#) (const MeshInfo &mi, indice owner)
Extract smoothness indicator from pre-calculated values.
- unordered_map< int, double > [GetSmoothnessIndicDeriv](#) (const MeshInfo &mi, const indice &owner)
- void [UpdateSmoothnessIndic](#) (const MeshInfo &mi)
Update smoothness indicator for entire reconstruction level.
- void [UpdateDerivSmoothnessIndic](#) (const MeshInfo &mi)
- double [Evaluate](#) (const MeshInfo &mi, const indice &owner, const vertex &point)
- double [Evaluate](#) (const MeshInfo &mi, const indice &owner, const vertex &point, const int &local)
- void [CheckStencils](#) () const
class members for checking and verification
- void [CheckStencilPolynomials](#) (const MeshInfo &mi, indice start)
Functions checking created stencil polynomials for a single stencil.
- void [PrintSmoothnessIndicator](#) (const MeshInfo &mi)
Print all smoothnessIndicator for all stencils belonging to this level.

5.4.1 Constructor & Destructor Documentation

5.4.1.1 singleLevelReconstruction() [1/2]

```
MLWENO::singleLevelReconstruction::singleLevelReconstruction ( ) [inline]
```

A constructor.

Default constructor for a single reconstruction level.

5.4.1.2 singleLevelReconstruction() [2/2]

```
singleLevelReconstruction::singleLevelReconstruction (
    int stencilSizeX,
    int stencilSizeY )
```

A constructor.

Costume constructor with given x and y stencil size. Create stencil of indice used in creating stencil polynomials

5.4.1.3 ~singleLevelReconstruction()

```
singleLevelReconstruction::~~singleLevelReconstruction ( )
```

A destructor.

Clear interior, single level polynomial and smoothness indicators

5.4.2 Member Function Documentation

5.4.2.1 CalculateSmoothnessIndic()

```
double singleLevelReconstruction::CalculateSmoothnessIndic (
    const MeshInfo & mi,
    indice owner )
```

Calculate smoothness indicator for the single stencil.

Directly calculate smoothness indicator of a given stencil Should not be called directly for computational efficiency

5.4.2.2 CheckExist()

```
bool MLWENO::singleLevelReconstruction::CheckExist (
    const MeshInfo & mi,
    indice owner ) const [inline]
```

5.4.2.3 CheckStencilPolynomials()

```
void singleLevelReconstruction::CheckStencilPolynomials (
    const MeshInfo & mi,
    indice start )
```

Functions checking created stencil polynomials for a single stencil.

5.4.2.4 CheckStencils()

```
void MLWENO::singleLevelReconstruction::CheckStencils ( ) const [inline]
```

class members for checking and verification

5.4.2.5 CreateStencilPolynomials()

```
void singleLevelReconstruction::CreateStencilPolynomials (
    const MeshInfo & mi )
```

! Two steps of creating stencil polynomials for a given reconstruction level. Step 1: Identify all the stencils covering entire computational domain. Step 2: Compute stencil polynomials for each stencil.

5.4.2.6 Evaluate() [1/2]

```
double singleLevelReconstruction::Evaluate (
    const MeshInfo & mi,
    const indice & owner,
    const vertex & point )
```

Evaluate polynomial

Evaluate at the given single reconstruction level

5.4.2.7 Evaluate() [2/2]

```
double singleLevelReconstruction::Evaluate (
    const MeshInfo & mi,
    const indice & owner,
    const vertex & point,
    const int & local )
```

Evaluate at the given single reconstruction level but evaluate individual stencil polynomials separately not the collapsed one.

5.4.2.8 GetScale() [1/2]

```
const double MLWENO::singleLevelReconstruction::GetScale (
    const MeshInfo & mi,
    const indice & owner ) [inline]
```

5.4.2.9 GetScale() [2/2]

```
const double MLWENO::singleLevelReconstruction::GetScale (
    int s ) [inline]
```

5.4.2.10 GetSizeX()

```
const int MLWENO::singleLevelReconstruction::GetSizeX ( ) const [inline]
```

5.4.2.11 GetSizeY()

```
const int MLWENO::singleLevelReconstruction::GetSizeY ( ) const [inline]
```

5.4.2.12 GetSmoothnessIndic()

```
double singleLevelReconstruction::GetSmoothnessIndic (
    const MeshInfo & mi,
    indice owner )
```

Extract smoothness indicator from pre-calculated values.

Directly extract pre-calculated smoothness indicator. Should always be the one to call when smoothness indicator is needed.

5.4.2.13 GetSmoothnessIndicDeriv()

```
unordered_map< int, double > singleLevelReconstruction::GetSmoothnessIndicDeriv (
    const MeshInfo & mi,
    const indice & owner )
```

5.4.2.14 PrintSmoothnessIndicator()

```
void singleLevelReconstruction::PrintSmoothnessIndicator (
    const MeshInfo & mi )
```

Print all smoothnessIndicator for all stencils belonging to this level.

5.4.2.15 UpdateDerivSmoothnessIndic()

```
void singleLevelReconstruction::UpdateDerivSmoothnessIndic (
    const MeshInfo & mi )
```

5.4.2.16 UpdateSmoothnessIndic()

```
void singleLevelReconstruction::UpdateSmoothnessIndic (
    const MeshInfo & mi )
```

Update smoothness indicator for entire reconstruction level.

Update smoothness indicators for the entire level.

The documentation for this class was generated from the following files:

- /home/renpo/Research/MantleSolver/src/mlweno/[reconstruction.h](#)
- /home/renpo/Research/MantleSolver/src/mlweno/[reconstruction.cpp](#)

5.5 stencil< T > Class Template Reference

```
#include <stencil.h>
```

Public Member Functions

- [stencil](#) ()
- [stencil](#) (size_t I)
- [stencil](#) (size_t I, size_t J)
- [stencil](#) (size_t I, size_t J, size_t K)
- [~stencil](#) ()
- void [SetStencil](#) (size_t I)
- void [SetStencil](#) (size_t I, size_t J)
- void [SetStencil](#) (size_t I, size_t J, size_t K)
- void [CreateStencil](#) ()
- const int [getSize](#) () const
- const int [getI](#) () const
- const int [getJ](#) () const
- const int [getK](#) () const
- void [setI](#) (size_t I) const
- void [setJ](#) (size_t J) const
- void [setK](#) (size_t K) const
- T & [operator\(\)](#) (size_t i)
- T & [operator\(\)](#) (size_t i, size_t j)
- T & [operator\(\)](#) (size_t i, size_t j, size_t k)
- T [operator\(\)](#) (size_t i) const
- T [operator\(\)](#) (size_t i, size_t j) const
- T [operator\(\)](#) (size_t i, size_t j, size_t k) const
- const vector< T > & [GetStencil](#) () const
- void [clearPtr](#) ()

5.5.1 Constructor & Destructor Documentation

5.5.1.1 stencil() [1/4]

```
template<class T >
stencil< T >::stencil ( ) [inline]
```

5.5.1.2 stencil() [2/4]

```
template<class T >
stencil< T >::stencil (
    size_t I ) [inline]
```

5.5.1.3 stencil() [3/4]

```
template<class T >
stencil< T >::stencil (
    size_t I,
    size_t J ) [inline]
```

5.5.1.4 stencil() [4/4]

```
template<class T >
stencil< T >::stencil (
    size_t I,
    size_t J,
    size_t K ) [inline]
```

5.5.1.5 ~stencil()

```
template<class T >
stencil< T >::~stencil ( ) [inline]
```

5.5.2 Member Function Documentation

5.5.2.1 clearPtr()

```
template<class T >
void stencil< T >::clearPtr ( ) [inline]
```

5.5.2.2 CreateStencil()

```
template<class T >
void stencil< T >::CreateStencil ( ) [inline]
```

5.5.2.3 getI()

```
template<class T >
const int stencil< T >::getI ( ) const [inline]
```

5.5.2.4 getJ()

```
template<class T >
const int stencil< T >::getJ ( ) const [inline]
```

5.5.2.5 getK()

```
template<class T >
const int stencil< T >::getK ( ) const [inline]
```

5.5.2.6 getSize()

```
template<class T >
const int stencil< T >::getSize ( ) const [inline]
```

5.5.2.7 GetStencil()

```
template<class T >
const vector< T > & stencil< T >::GetStencil ( ) const [inline]
```

5.5.2.8 operator()() [1/6]

```
template<class T >
T & stencil< T >::operator() (
    size_t i ) [inline]
```

5.5.2.9 operator() [2/6]

```
template<class T >
T stencil< T >::operator() (
    size_t i ) const [inline]
```

5.5.2.10 operator() [3/6]

```
template<class T >
T & stencil< T >::operator() (
    size_t i,
    size_t j ) [inline]
```

5.5.2.11 operator() [4/6]

```
template<class T >
T stencil< T >::operator() (
    size_t i,
    size_t j ) const [inline]
```

5.5.2.12 operator() [5/6]

```
template<class T >
T & stencil< T >::operator() (
    size_t i,
    size_t j,
    size_t k ) [inline]
```

5.5.2.13 operator() [6/6]

```
template<class T >
T stencil< T >::operator() (
    size_t i,
    size_t j,
    size_t k ) const [inline]
```

5.5.2.14 setI()

```
template<class T >
void stencil< T >::setI (
    size_t I ) const [inline]
```

5.5.2.15 setJ()

```
template<class T >
void stencil< T >::setJ (
    size_t J ) const [inline]
```

5.5.2.16 setK()

```
template<class T >
void stencil< T >::setK (
    size_t K ) const [inline]
```

5.5.2.17 SetStencil() [1/3]

```
template<class T >
void stencil< T >::SetStencil (
    size_t I ) [inline]
```

5.5.2.18 SetStencil() [2/3]

```
template<class T >
void stencil< T >::SetStencil (
    size_t I,
    size_t J ) [inline]
```

5.5.2.19 SetStencil() [3/3]

```
template<class T >
void stencil< T >::SetStencil (
    size_t I,
    size_t J,
    size_t K ) [inline]
```

The documentation for this class was generated from the following file:

- /home/renpo/Research/MantleSolver/src/mlweno/[stencil.h](#)

5.6 tensorProductPoly::stencilPolynomial Class Reference

```
#include <polynomial.h>
```

Public Member Functions

- [stencilPolynomial](#) ()
- [stencilPolynomial](#) (const indice &start, const vertex ¢er)
- [stencilPolynomial](#) (const indice &start, const vertex ¢er, const vector< indice > &targetCell)
- [~stencilPolynomial](#) ()
- void [SetStencilPolynomials](#) (const MeshInfo &mi, const [stencil](#)< indice > &stencilIndice)
- void [SetUpScale](#) (const MeshInfo &mi, const vector< indice > &targetCell)
- void [SetUpScale](#) (const MeshInfo &mi, const [stencil](#)< indice > &stencilIndice)
- const double [GetScale](#) () const
- double [eval](#) (const double x, const double y) const
- double [eval](#) (const vertex &P) const
- double [operator\(\)](#) (const double x, const double y) const
- double [operator\(\)](#) (const vertex &P) const
- double [eval](#) (double x, double y, int poly) const
- double [eval](#) (vertex P, int poly) const
- double [operator\(\)](#) (double x, double y, int poly) const
- double [operator\(\)](#) (vertex P, int poly) const
- void [printCoef](#) ()
- void [printCoef](#) (int s)
- void [SetCollapsePolyn](#) (const MeshInfo &mi, const [stencil](#)< indice > &stencilIndice)
- double [GetSmoothIndic](#) (const MeshInfo &mi, const [stencil](#)< indice > &stencilIndice)
- unordered_map< int, double > [GetDerivSmoothIndic](#) (const MeshInfo &mi, const [stencil](#)< indice > &stencilIndice)
- const int [GetOrderX](#) () const
- const int [GetOrderY](#) () const

5.6.1 Constructor & Destructor Documentation

5.6.1.1 stencilPolynomial() [1/3]

```
tensorProductPoly::stencilPolynomial::stencilPolynomial ( ) [inline]
```

5.6.1.2 stencilPolynomial() [2/3]

```
stencilPolynomial::stencilPolynomial (
    const indice & start,
    const vertex & center )
```


5.6.1.3 stencilPolynomial() [3/3]

```
stencilPolynomial::stencilPolynomial (
    const indice & start,
    const vertex & center,
    const vector< indice > & targetCell )
```

5.6.1.4 ~stencilPolynomial()

```
tensorProductPoly::stencilPolynomial::~~stencilPolynomial ( ) [inline]
```

5.6.2 Member Function Documentation**5.6.2.1 eval()** [1/4]

```
double stencilPolynomial::eval (
    const double x,
    const double y ) const
```

Shift input point with respect to stencil center

5.6.2.2 eval() [2/4]

```
double tensorProductPoly::stencilPolynomial::eval (
    const vertex & P ) const [inline]
```

5.6.2.3 eval() [3/4]

```
double stencilPolynomial::eval (
    double x,
    double y,
    int poly ) const
```

Shift input point with respect to the stencil center

5.6.2.4 eval() [4/4]

```
double tensorProductPoly::stencilPolynomial::eval (
    vertex P,
    int poly ) const [inline]
```

5.6.2.5 GetDerivSmoothIndic()

```
unordered_map< int, double > stencilPolynomial::GetDerivSmoothIndic (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndices )
```

5.6.2.6 GetOrderX()

```
const int tensorProductPoly::stencilPolynomial::GetOrderX ( ) const [inline]
```

5.6.2.7 GetOrderY()

```
const int tensorProductPoly::stencilPolynomial::GetOrderY ( ) const [inline]
```

5.6.2.8 GetScale()

```
const double tensorProductPoly::stencilPolynomial::GetScale ( ) const [inline]
```

5.6.2.9 GetSmoothIndic()

```
double stencilPolynomial::GetSmoothIndic (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndices )
```

5.6.2.10 operator>() [1/4]

```
double tensorProductPoly::stencilPolynomial::operator() (
    const double x,
    const double y ) const [inline]
```

5.6.2.11 operator>() [2/4]

```
double tensorProductPoly::stencilPolynomial::operator() (
    const vertex & P ) const [inline]
```

5.6.2.12 operator>() [3/4]

```
double tensorProductPoly::stencilPolynomial::operator() (
    double x,
    double y,
    int poly ) const [inline]
```

5.6.2.13 operator>() [4/4]

```
double tensorProductPoly::stencilPolynomial::operator() (
    vertex P,
    int poly ) const [inline]
```

5.6.2.14 printCoef() [1/2]

```
void stencilPolynomial::printCoef ( )
```

5.6.2.15 printCoef() [2/2]

```
void stencilPolynomial::printCoef (
    int s )
```

5.6.2.16 SetCollapsePolyn()

```
void stencilPolynomial::SetCollapsePolyn (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndice )
```

5.6.2.17 SetStencilPolynomials()

```
void stencilPolynomial::SetStencilPolynomials (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndice )
```

5.6.2.18 `SetUpScale()` [1/2]

```
void tensorProductPoly::stencilPolynomial::SetUpScale (
    const MeshInfo & mi,
    const stencil< indice > & stencilIndices ) [inline]
```

5.6.2.19 `SetUpScale()` [2/2]

```
void tensorProductPoly::stencilPolynomial::SetUpScale (
    const MeshInfo & mi,
    const vector< indice > & targetCell ) [inline]
```

The documentation for this class was generated from the following files:

- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h](#)
- [/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp](#)

Chapter 6

File Documentation

6.1 `/home/renpo/Research/MantleSolver/src/mlweno/polynomial.cpp` File Reference

```
#include "polynomial.h"
```

6.2 `/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h` File Reference

```
#include "util.h"  
#include "stencil.h"
```

Classes

- class `tensorProductPoly::basePolynomial`
- class `tensorProductPoly::stencilPolynomial`

Namespaces

- namespace `tensorProductPoly`

6.3 polynomial.h

[Go to the documentation of this file.](#)

```

00001 #ifndef POLYNOMIAL_H_
00002 #define POLYNOMIAL_H_
00003
00004 #include "util.h"
00005 #include "stencil.h"
00006
00007 namespace tensorProductPoly{
00008     // Tensor product 2D polynomial for stencil basis
00009     class basePolynomial {
00010     public:
00011         basePolynomial() {};
00012         basePolynomial(const int maxDegree[2]);
00013         basePolynomial(const int maxDegree[2], double* coef);
00014         ~basePolynomial();
00015
00016         void setMaxDegree(const int maxDegree[2]);
00017         void setCoef(double* coef);
00018
00019         double* getCoef() const;
00020
00021         // Return the size of the polynomial
00022         int getSize() const {return maxDegree[0]*maxDegree[1];};
00023
00024         // Return the size of degree x as I
00025         int getI() const {return maxDegree[0];};
00026         // Return the size of degree y as J
00027         int getJ() const {return maxDegree[1];};
00028
00029         // Evaluation of point value for a given polynomial
00030         double eval(double x, double y) const;
00031         double eval(vertex P) const {return eval(P[0],P[1]);};
00032         double operator() (double x, double y) const {return eval(x,y);};
00033         double operator() (vertex P) const {return eval(P);};
00034
00035         // Evaluation of derivative value for a given polynomial
00036         double der(int derX, int derY, double x, double y) const;
00037         double der(int derX, int derY, vertex P) const {return der(derX, derY, P[0], P[1]);};
00038
00039         // Print coefficients out
00040         void printCoef() const;
00041
00042         double getCoef(int i) const;
00043
00044     private:
00045         int maxDegree_[2] = {-1,-1};
00046         double* coef_ = nullptr;
00047     };
00048
00049 // stencil polynomials used for weno reconstruction
00050 class stencilPolynomial {
00051 public:
00052     stencilPolynomial() {};
00053     stencilPolynomial(const indice& start, const vertex& center);
00054     stencilPolynomial(const indice& start, const vertex& center,
00055                     const vector<indice>& targetCell);
00056     ~stencilPolynomial() {stencilPolyn_.clearPtr();
00057                         delete collapsePolyn_};
00058
00059     void SetStencilPolynomials(const MeshInfo& mi,
00060                             const stencil <indice>& stencilIndice);
00061
00062     void SetUpScale(const MeshInfo& mi, const vector<indice>& targetCell)
00063         {SetTargetCell_(targetCell);ComputeCellBasedScale_(mi);};
00064     void SetUpScale(const MeshInfo& mi, const stencil <indice>& stencilIndice)
00065         {ComputeStencilBasedScale_(mi, stencilIndice);};
00066
00067     const double GetScale() const {return scale_};};
00068
00069     double eval(const double x, const double y) const;
00070     double eval(const vertex& P) const {return eval(P[0],P[1]);};
00071     double operator() (const double x, const double y) const {return eval(x,y);};
00072     double operator() (const vertex& P) const {return eval(P);};
00073
00074     // Evaluation of point value for a given polynomial.
00075     // Separately without using collapsed polynomial.
00076     double eval(double x, double y, int poly) const;
00077     double eval(vertex P, int poly) const {return eval(P[0],P[1],poly);};
00078     double operator() (double x, double y, int poly) const {return eval(x,y,poly);};
00079     double operator() (vertex P, int poly) const {return eval(P,poly);};
00080
00081

```

```

00082
00083     // Check basis polynomial coefficients
00084     void printCoef();
00085     void printCoef(int s);
00086
00087     // Set collapse polynomial
00088     void SetCollapsePolyn(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00089
00090     double GetSmoothIndic(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00091
00092     unordered_map<int, double> GetDerivSmoothIndic(const MeshInfo& mi, const stencil<indice>&
stencilIndice);
00093
00094     const int GetOrderX() const { return stencilPolyn_.getI();};
00095     const int GetOrderY() const { return stencilPolyn_.getJ();};
00096
00097     private:
00098
00099     vertex center_ = {0.0,0.0};
00100     indice start_ = {-1,-1};
00101
00102     void SetTargetCell_(const vector<indice>& targetCell);
00103     vector<indice> targetCell_;
00104     double scale_ = -1.0;
00105
00106     double coef_ = 1.0;
00107
00108     // Calculate scale based on the information of a single cell
00109     void ComputeCellBasedScale_(const MeshInfo& mi);
00110     // Calculate scale based on the value of a given stencil
00111     void ComputeStencilBasedScale_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00112
00113     stencil <basePolynomial*> stencilPolyn_;
00114
00115     void SetCollapsePolyn_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00116     basePolynomial* collapsePolyn_ = nullptr;
00117
00118     // Create polynomial smoothness indicator
00119     void CreateXi_();
00120     vector<double> Xi_;
00121
00122     int maxR_;
00123     int minR_;
00124
00129     void EvalSmoothIndic_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00130     void EvalDerivSmoothnessIndic_(const MeshInfo& mi, const stencil <indice>& stencilIndice);
00131
00132     double smoothnessIndic_ = -1;
00133     unordered_map<int, double> derivSmoothnessIndic_;
00134
00135 };
00136
00137 // End of using name space MLWENO
00138 }
00139
00140 #endif

```

6.4 /home/renpo/Research/MantleSolver/src/mlweno/reconstruction.cpp File Reference

```
#include "reconstruction.h"
```

6.5 /home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h File Reference

```
#include "polynomial.h"
#include <map>
```

Classes

- class [MLWENO::singleLevelReconstruction](#)
- class [MLWENO::MLWENOPrepare](#)
- class [MLWENO::multiLevelReconstruction](#)

Namespaces

- namespace [MLWENO](#)
Weno reconstruction with a given stencil level.

6.6 reconstruction.h

[Go to the documentation of this file.](#)

```

00001 #ifndef RECONSTRUCTION_H_
00002 #define RECONSTRUCTION_H_
00003
00004 //\sreconstruction.h slreconstruction.h
00005
00006
00007 #include "polynomial.h"
00008 #include <map>
00009
00010 namespace MLWENO{
00011
00012     class singleLevelReconstruction {
00013     public:
00014
00015         singleLevelReconstruction() {};
00016
00017         singleLevelReconstruction(int stencilSizeX, int stencilSizeY);
00018
00019         ~singleLevelReconstruction();
00020
00021         // =====
00022         void CreateStencilPolynomials(const MeshInfo& mi);
00023
00024         bool CheckExist(const MeshInfo& mi, indice owner) const {return
interior_.count(FlatIndic(mi,owner));};
00025
00026         const int GetSizeX() const {return stencilSizeX_};
00027         const int GetSizeY() const {return stencilSizeY_};
00028
00029         const double GetScale(int s) {return singleLevel_[s]->GetScale();};
00030         const double GetScale(const MeshInfo& mi, const indice& owner) {return
singleLevel_[FlatIndic(mi,owner)]->GetScale();};
00031
00032         double CalculateSmoothnessIndic(const MeshInfo& mi, indice owner);
00033
00034         double GetSmoothnessIndic(const MeshInfo& mi, indice owner);
00035
00036         unordered_map<int, double> GetSmoothnessIndicDeriv(const MeshInfo& mi, const indice&
owner);
00037
00038         void UpdateSmoothnessIndic(const MeshInfo& mi);
00039
00040         void UpdateDerivSmoothnessIndic(const MeshInfo& mi);
00041
00042         double Evaluate(const MeshInfo& mi, const indice& owner, const vertex& point);
00043
00044         double Evaluate(const MeshInfo& mi, const indice& owner, const vertex& point, const int&
local);
00045
00046         // =====
00047         void CheckStencils() const {cout<< "Constructed "<< interior_.size() << " stencils with the
size of " << stencilSizeX_ << " " << stencilSizeY_ << endl;};
00048         void CheckStencilPolynomials(const MeshInfo& mi, indice start);
00049         void PrintSmoothnessIndicator(const MeshInfo& mi);
00050
00051     private:
00052
00053         int stencilSizeX_ = -1;
00054         int stencilSizeY_ = -1;
00055
00056
00057
00058
00059
00060
00061
00062
00063
00064
00065
00066
00067
00068
00069
00070
00071
00072
00073
00074
00075
00076

```



```

00077         stencil <indice> stencilIndice_;
00078         unordered_set<int> interior_;
00079         map<int, double> smoothnessIndic_;
00080         unordered_map<int, unordered_map<int, double> smoothnessIndicDeriv_;
00081         map<int, tensorProductPoly::stencilPolynomial*> singleLevel_;
00082
00083         void IdentifyInteriorCell_(const MeshInfo& mi);
00084
00085         const vertex ComputeStencilCenter_(const MeshInfo& mi, int flat);
00086
00087         void ComputeStencilPolyn_(const MeshInfo& mi);
00088     };
00089
00090     class MLWENOPprepare {
00091     public:
00092         MLWENOPprepare() {};
00093
00094         ~MLWENOPprepare();
00095
00096         void AddLevel(const MeshInfo& mi, const int& stencilSizeX,
00097                     const int& stencilSizeY);
00098
00099         void UpdateSmoothnessIndic(const MeshInfo& mi);
00100
00101         void UpdateDerivSmoothnessIndic(const MeshInfo& mi);
00102
00103         void PrintInfo();
00104
00105     private:
00106         friend class multiLevelReconstruction;
00107
00108         std::unordered_map<std::string, singleLevelReconstruction *> allLevels_;
00109     };
00110
00111     class multiLevelReconstruction {
00112     public:
00113         multiLevelReconstruction() {};
00114
00115         multiLevelReconstruction(const MeshInfo& mi, int stencilSizeX, int stencilSizeY,
00116                                 vector<indice> brm)
00117         {AddLevel(mi, stencilSizeX, stencilSizeY, brm);};
00118
00119         multiLevelReconstruction(const MeshInfo& mi, int* stencilSize, vector<indice> brm)
00120         {AddLevel(mi, stencilSize, brm);};
00121
00122         multiLevelReconstruction(const MeshInfo& mi, vector<int*> stencilSizes,
00123                                 vector<vector<indice> brms)
00124         {AddLevel(mi, stencilSizes, brms);};
00125
00126         ~multiLevelReconstruction() {Clear();};
00127
00128         void AddLevel(const MeshInfo& mi, int stencilSizeX, int stencilSizeY, vector<indice> brm);
00129         void AddLevel(const MeshInfo& mi, int* stencilSize, vector<indice> brm)
00130         {AddLevel(mi, stencilSize[0], stencilSize[1], brm);};
00131         void AddLevel(const MeshInfo& mi, vector<int*> stencilSizes, vector<vector<indice> brms)
00132         {for (int i=0; i<stencilSizes.size(); i++){
00133             AddLevel(mi, stencilSizes[i], brms[i]);};};
00134
00135         void SeparateBoundaryLayer(const MeshInfo& mi, const int& layerSize);
00136         void SeparateBoundaryLayer(const MeshInfo& mi);
00137
00138         void SeparateBoundaryLayer(const MeshInfo& mi, const int& layerSize,
00139                                     const unordered_set<std::string>& additionalLevels);
00140
00141         const unordered_set<int>& GetboundaryCells() const
00142         {return boundaryCells_;};
00143         const unordered_set<int>& GetinteriorCells() const
00144         {return interiorCells_;};
00145
00146         void AssignboundaryCells (const unordered_set<int>& boundaryCells)
00147         {boundaryCells_ = boundaryCells;};
00148         void AssigninteriorCells (const unordered_set<int>& interiorCells)
00149         {interiorCells_ = interiorCells;};
00150
00151         const unordered_set<std::string>& GetboundaryLevels() const
00152         {return boundaryLevels_;};
00153         const unordered_set<std::string>& GetinteriorLevels() const
00154         {return interiorLevels_;};
00155
00156         void AssignboundaryLevels (const unordered_set<std::string>& boundaryLevels)
00157         {boundaryLevels_ = boundaryLevels;};
00158         void AssigninteriorLevels (const unordered_set<std::string>& interiorLevels)
00159         {interiorLevels_ = interiorLevels;};
00160
00161         void UpdateNonLinearWgts(const MeshInfo& mi, const int stage);

```

```

00215
00219         void UpdateNonLinearWgts(const MeshInfo& mi);
00220
00221         void UpdateNonLinearWgtsAndDerivs(const MeshInfo& mi);
00222
00230         void ModifyReconstMethod(std::string key, vector<indice> newReconstMethod);
00231         void ModifyReconstMethod(int stencilSizeX, int stencilSizeY, vector<indice>
newReconstMethod)
00232         {std::string key = '(' + std::to_string(stencilSizeX) + ',' + std::to_string(stencilSizeY)
+ ')';
00233             ModifyReconstMethod(key,newReconstMethod);};
00234
00238         void SelectWenoReconstLevel(const unordered_set<std::string>& keys);
00239
00243         void SelectWenoReconstLevel(const unordered_set<std::string>& keys,
const MLWENOPrepare& mlpPtr);
00244
00245         double EvaluateMLWENO(const MeshInfo& mi, vertex point, indice cell) const;
00249
00250         unordered_map<int, double> EvaluateDerivMLWENO(const MeshInfo& mi,
const vertex& point,
00256             const indice& global) const;
00257
00258         unordered_map<int, double> EvaluateDerivMLWENOAdd(const MeshInfo& mi,
const vertex& point,
00259             const indice& global) const;
00260
00261         const double GetScale(std::string level, int s)
00266         {return reconstLevels_[level]->GetScale(s);};
00267         const double GetScale(std::string level, const MeshInfo& mi, const indice& owner)
00268         {return reconstLevels_[level]->GetScale(mi,owner);};
00269
00270         void GetInfo();
00272         void PrintBoundaryLayer(const MeshInfo& mi);
00273         void PrintSmoothnessIndicator(const MeshInfo& mi);
00274         void PrintNonLinearWgts(const MeshInfo& mi);
00275
00276         void Clear();
00283     private:
00284
00285         const double eps0_ = 0.01;
00286
00287         std::string lowestLevel_;
00288         std::string highestLevel_;
00289
00290         map<std::string, singleLevelReconstruction *> reconstLevels_;
00291         map<std::string, vector<indice> > reconstMethods_;
00292
00293         unordered_set<int> interiorCells_;
00294         unordered_set<int> boundaryCells_;
00295
00296         unordered_set<std::string> wenoLevels_;
00297         unordered_set<std::string> interiorLevels_;
00298         unordered_set<std::string> boundaryLevels_;
00299
00300         int totalLevels_ = 0;
00301
00302         unordered_map<int, unordered_map<std::string, unordered_map<int,double>> nonLinearWgts_;
00303
00304         // The most outside map creates map from global index of cells to a set of non linear
00307         weights
00308         // The inner map from levels (std::string) to a set of non linear weights (within the same
level)
00309         // The most inside map connects derivatives (a map mapping with cells in the chosen
stencil).
00310         unordered_map<int, unordered_map<std::string, unordered_map<int,unordered_map<int,double>>>
derivNonLinearWgts_;
00311
00316         void SeparateReconstMethods_();
00317
00318         void SeparateReconstMethods_(const unordered_set<std::string>& additionalLevels);
00319
00324         void UpdateOneStageNonLinearWgts_(const MeshInfo& mi, int flatGlobal,
const unordered_set<std::string>& levels);
00325
00326         void UpdateTwoStageNonLinearWgts_(const MeshInfo& mi, int flatGlobal,
const unordered_set<std::string>& levels);
00327
00328         void UpdateNonLinearWgtsAndDerivs_(const MeshInfo& mi, int flatGlobal,
const unordered_set<std::string>& levels);
00329
00334         vector< map<int, int> > etaBias_;
00335
00336         bool prepare_ = false;
00338     };
00339 }
00343 }
00344
00345 #endif

```

6.7 /home/renpo/Research/MantleSolver/src/mlweno/stencil.h File Reference

```
#include "util.h"
```

Classes

- class [stencil< T >](#)

Enumerations

- enum [stencilType](#) { [cellCentered](#), [edgeCentered](#), [vertexCentered](#) }

6.7.1 Enumeration Type Documentation

6.7.1.1 stencilType

```
enum stencilType
```

Enumerator

cellCentered	
edgeCentered	
vertexCentered	

6.8 stencil.h

[Go to the documentation of this file.](#)

```
00001 #ifndef STENCIL_H_
00002 #define STENCIL_H_
00003
00004 /*
00005  *Define multi-level stencils for weno reconstruction
00006  *Designed for cartesian logically rectangular mesh.
00007  */
00008
00009 #include "util.h"
00010
00011
00012 enum stencilType {cellCentered, edgeCentered, vertexCentered};
00013
00014 template <class T>
00015 class stencil{
00016     public:
00017         stencil() {};
00018         stencil(size_t I):                                I_(I), stencilSize_(I)
00019         {stencil_.resize(stencilSize_);};
00019         stencil(size_t I, size_t J):                      I_(I), J_(J), stencilSize_(I*J)
00020         {stencil_.resize(stencilSize_);};
```

```

00020         stencil(size_t I, size_t J, size_t K): I_(I), J_(J), K_(K), stencilSize_(I*J*K)
00021     {stencil_.resize(stencilSize_);};
00021
00022     ~stencil() {};;
00023
00024     void SetStencil(size_t I) {I_ = I; stencilSize_ = I; stencil_.resize(stencilSize_);}
00025     void SetStencil(size_t I, size_t J) {I_ = I; J_ = J; stencilSize_ = I*J;
00026         stencil_.resize(stencilSize_);}
00026     void SetStencil(size_t I, size_t J, size_t K) {I_ = I; J_ = J; K_ = K; stencilSize_ = I*J*K;
00027         stencil_.resize(stencilSize_);}
00027
00028     void CreateStencil() {stencil_.resize(stencilSize_);}
00029
00030     const int getSize() const {return stencilSize_};;
00031
00032     const int getI() const {return I_};;
00033     const int getJ() const {return J_};;
00034     const int getK() const {return K_};;
00035
00036     void setI(size_t I) const {I_ = I};;
00037     void setJ(size_t J) const {J_ = J};;
00038     void setK(size_t K) const {K_ = K};;
00039
00040     T &operator()(size_t i){
00041         return stencil_[i];
00042     }
00043
00044     T &operator()(size_t i, size_t j){
00045         return stencil_[i+j*I_];
00046     }
00047
00048     T &operator()(size_t i, size_t j, size_t k){
00049         return stencil_[i+j*I_+k*I_*J_];
00050     }
00051
00052     T operator()(size_t i) const{
00053         return stencil_[i];
00054     }
00055
00056     T operator()(size_t i, size_t j) const{
00057         return stencil_[i+j*I_];
00058     }
00059
00060     T operator()(size_t i, size_t j, size_t k) const{
00061         return stencil_[i+j*I_+k*I_*J_];
00062     }
00063
00064     const vector<T>& GetStencil() const {return stencil_};
00065
00066     // Call it when T is pointer
00067     void clearPtr() {if (std::is_pointer<T>::value){
00068         std::for_each(stencil_.begin(), stencil_.end(), delete_pointed_to<T>);}}
00069
00070     private:
00071         size_t I_;
00072         size_t J_;
00073         size_t K_;
00074         size_t stencilSize_;
00075         vector<T> stencil_;
00076 };
00077
00078 #endif

```

Index

/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h, 37
/home/renpo/Research/MantleSolver/src/mlweno/polynomial.h, 37, 38
/home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h, 39
/home/renpo/Research/MantleSolver/src/mlweno/reconstruction.h, 39, 40
/home/renpo/Research/MantleSolver/src/mlweno/stencil.h, 43
~MLWENOPPrepare
 MLWENO::MLWENOPPrepare, 13
~basePolynomial
 tensorProductPoly::basePolynomial, 10
~multiLevelReconstruction
 MLWENO::multiLevelReconstruction, 16
~singleLevelReconstruction
 MLWENO::singleLevelReconstruction, 24
~stencil
 stencil< T >, 28
~stencilPolynomial
 tensorProductPoly::stencilPolynomial, 33
AddLevel
 MLWENO::MLWENOPPrepare, 13
 MLWENO::multiLevelReconstruction, 16, 17
AssignboundaryCells
 MLWENO::multiLevelReconstruction, 17
AssignboundaryLevels
 MLWENO::multiLevelReconstruction, 17
AssigninteriorCells
 MLWENO::multiLevelReconstruction, 17
AssigninteriorLevels
 MLWENO::multiLevelReconstruction, 18
basePolynomial
 tensorProductPoly::basePolynomial, 9, 10
CalculateSmoothnessIndic
 MLWENO::singleLevelReconstruction, 24
cellCentered
 stencil.h, 43
CheckExist
 MLWENO::singleLevelReconstruction, 24
CheckStencilPolynomials
 MLWENO::singleLevelReconstruction, 24
CheckStencils
 MLWENO::singleLevelReconstruction, 24
Clear
 MLWENO::multiLevelReconstruction, 18
clearPtr
 stencil< T >, 28
CreateStencil
 stencil< T >, 28
CreateStencilPolynomials
 MLWENO::singleLevelReconstruction, 25
der
 tensorProductPoly::basePolynomial, 10
edgeCentered
 stencil.h, 43
eval
 tensorProductPoly::basePolynomial, 10
 tensorProductPoly::stencilPolynomial, 33
Evaluate
 MLWENO::singleLevelReconstruction, 25
EvaluateDerivMLWENO
 MLWENO::multiLevelReconstruction, 18
EvaluateDerivMLWENOAdd
 MLWENO::multiLevelReconstruction, 18
EvaluateMLWENO
 MLWENO::multiLevelReconstruction, 18
GetboundaryCells
 MLWENO::multiLevelReconstruction, 19
GetboundaryLevels
 MLWENO::multiLevelReconstruction, 19
getCoef
 tensorProductPoly::basePolynomial, 11
GetDerivSmoothIndic
 tensorProductPoly::stencilPolynomial, 33
getI
 stencil< T >, 29
 tensorProductPoly::basePolynomial, 11
GetInfo
 MLWENO::multiLevelReconstruction, 19
GetinteriorCells
 MLWENO::multiLevelReconstruction, 19
GetinteriorLevels
 MLWENO::multiLevelReconstruction, 19
getJ
 stencil< T >, 29
 tensorProductPoly::basePolynomial, 11
getK
 stencil< T >, 29
GetOrderX
 tensorProductPoly::stencilPolynomial, 34
GetOrderY
 tensorProductPoly::stencilPolynomial, 34

- GetScale
 - MLWENO::multiLevelReconstruction, 20
 - MLWENO::singleLevelReconstruction, 25
 - tensorProductPoly::stencilPolynomial, 34
- getSize
 - stencil < T >, 29
 - tensorProductPoly::basePolynomial, 11
- GetSizeX
 - MLWENO::singleLevelReconstruction, 25
- GetSizeY
 - MLWENO::singleLevelReconstruction, 26
- GetSmoothIndic
 - tensorProductPoly::stencilPolynomial, 34
- GetSmoothnessIndic
 - MLWENO::singleLevelReconstruction, 26
- GetSmoothnessIndicDeriv
 - MLWENO::singleLevelReconstruction, 26
- GetStencil
 - stencil < T >, 29
- MLWENO, 7
- MLWENO::MLWENOPrepare, 12
 - ~MLWENOPrepare, 13
 - AddLevel, 13
 - MLWENOPrepare, 13
 - multiLevelReconstruction, 14
 - PrintInfo, 13
 - UpdateDerivSmoothnessIndic, 13
 - UpdateSmoothnessIndic, 14
- MLWENO::multiLevelReconstruction, 14
 - ~multiLevelReconstruction, 16
 - AddLevel, 16, 17
 - AssignboundaryCells, 17
 - AssignboundaryLevels, 17
 - AssigninteriorCells, 17
 - AssigninteriorLevels, 18
 - Clear, 18
 - EvaluateDerivMLWENO, 18
 - EvaluateDerivMLWENOAdd, 18
 - EvaluateMLWENO, 18
 - GetboundaryCells, 19
 - GetboundaryLevels, 19
 - GetInfo, 19
 - GetinteriorCells, 19
 - GetinteriorLevels, 19
 - GetScale, 20
 - ModifyReconstMethod, 20
 - multiLevelReconstruction, 15, 16
 - PrintBoundaryLayer, 20
 - PrintNonLinearWgts, 20
 - PrintSmoothnessIndicator, 21
 - SelectWenoReconstLevel, 21
 - SeparateBoundaryLayer, 21, 22
 - UpdateNonLinearWgts, 22
 - UpdateNonLinearWgtsAndDerivs, 22
- MLWENO::singleLevelReconstruction, 23
 - ~singleLevelReconstruction, 24
 - CalculateSmoothnessIndic, 24
 - CheckExist, 24
 - CheckStencilPolynomials, 24
 - CheckStencils, 24
 - CreateStencilPolynomials, 25
 - Evaluate, 25
 - GetScale, 25
 - GetSizeX, 25
 - GetSizeY, 26
 - GetSmoothnessIndic, 26
 - GetSmoothnessIndicDeriv, 26
 - PrintSmoothnessIndicator, 26
 - singleLevelReconstruction, 23
 - UpdateDerivSmoothnessIndic, 26
 - UpdateSmoothnessIndic, 26
- MLWENOPrepare
 - MLWENO::MLWENOPrepare, 13
- ModifyReconstMethod
 - MLWENO::multiLevelReconstruction, 20
- multiLevelReconstruction
 - MLWENO::MLWENOPrepare, 14
 - MLWENO::multiLevelReconstruction, 15, 16
- operator()
 - stencil < T >, 29, 30
 - tensorProductPoly::basePolynomial, 11
 - tensorProductPoly::stencilPolynomial, 34, 35
- PrintBoundaryLayer
 - MLWENO::multiLevelReconstruction, 20
- printCoef
 - tensorProductPoly::basePolynomial, 12
 - tensorProductPoly::stencilPolynomial, 35
- PrintInfo
 - MLWENO::MLWENOPrepare, 13
- PrintNonLinearWgts
 - MLWENO::multiLevelReconstruction, 20
- PrintSmoothnessIndicator
 - MLWENO::multiLevelReconstruction, 21
 - MLWENO::singleLevelReconstruction, 26
- SelectWenoReconstLevel
 - MLWENO::multiLevelReconstruction, 21
- SeparateBoundaryLayer
 - MLWENO::multiLevelReconstruction, 21, 22
- setCoef
 - tensorProductPoly::basePolynomial, 12
- SetCollapsePolyn
 - tensorProductPoly::stencilPolynomial, 35
- setI
 - stencil < T >, 30
- setJ
 - stencil < T >, 31
- setK
 - stencil < T >, 31
- setMaxDegree
 - tensorProductPoly::basePolynomial, 12
- SetStencil
 - stencil < T >, 31
- SetStencilPolynomials
 - tensorProductPoly::stencilPolynomial, 35

- SetUpScale
 - tensorProductPoly::stencilPolynomial, [35](#), [36](#)
- singleLevelReconstruction
 - MLWENO::singleLevelReconstruction, [23](#)
- stencil
 - stencil< T >, [27](#), [28](#)
- stencil< T >, [27](#)
 - ~stencil, [28](#)
 - clearPtr, [28](#)
 - CreateStencil, [28](#)
 - getI, [29](#)
 - getJ, [29](#)
 - getK, [29](#)
 - getSize, [29](#)
 - GetStencil, [29](#)
 - operator(), [29](#), [30](#)
 - setI, [30](#)
 - setJ, [31](#)
 - setK, [31](#)
 - SetStencil, [31](#)
 - stencil, [27](#), [28](#)
- stencil.h
 - cellCentered, [43](#)
 - edgeCentered, [43](#)
 - stencilType, [43](#)
 - vertexCentered, [43](#)
- stencilPolynomial
 - tensorProductPoly::stencilPolynomial, [32](#)
- stencilType
 - stencil.h, [43](#)
- tensorProductPoly, [7](#)
- tensorProductPoly::basePolynomial, [9](#)
 - ~basePolynomial, [10](#)
 - basePolynomial, [9](#), [10](#)
 - der, [10](#)
 - eval, [10](#)
 - getCoef, [11](#)
 - getI, [11](#)
 - getJ, [11](#)
 - getSize, [11](#)
 - operator(), [11](#)
 - printCoef, [12](#)
 - setCoef, [12](#)
 - setMaxDegree, [12](#)
- tensorProductPoly::stencilPolynomial, [32](#)
 - ~stencilPolynomial, [33](#)
 - eval, [33](#)
 - GetDerivSmoothIndic, [33](#)
 - GetOrderX, [34](#)
 - GetOrderY, [34](#)
 - GetScale, [34](#)
 - GetSmoothIndic, [34](#)
 - operator(), [34](#), [35](#)
 - printCoef, [35](#)
 - SetCollapsePolyn, [35](#)
 - SetStencilPolynomials, [35](#)
 - SetUpScale, [35](#), [36](#)
 - stencilPolynomial, [32](#)
- UpdateDerivSmoothnessIndic
 - MLWENO::MLWENOPrepare, [13](#)
 - MLWENO::singleLevelReconstruction, [26](#)
- UpdateNonLinearWgts
 - MLWENO::multiLevelReconstruction, [22](#)
- UpdateNonLinearWgtsAndDerivs
 - MLWENO::multiLevelReconstruction, [22](#)
- UpdateSmoothnessIndic
 - MLWENO::MLWENOPrepare, [14](#)
 - MLWENO::singleLevelReconstruction, [26](#)
- vertexCentered
 - stencil.h, [43](#)